FIAP ON

COLHEITA DE DADOS E INSIGHTS - DADOS VALIOSOS E MADUROS

A MAGIA DA MATEMÁTICA NO PYTHON

3

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Representação de vetores e matrizes.....8
Figura 2 - Anatomia dos indexadores.....11
Figura 3 - Formalização do produto matricial 14
Figura 4 - *Array* armazenado em arquivo texto 17
Figura 5 - Imagem antes e depois do processamento — autorretrato de Van Gogh 20

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de declaração da biblioteca 7
Código-fonte 2 - Exemplo de criação de *arrays* a partir de uma lista 9
Código-fonte 3 - Exemplo de criação de *arrays* a partir do formato 9
Código-fonte 4 - Exemplo de criação de *arrays* com distribuições específicas 9
Código-fonte 5 - Acesso aos elementos de um *array* 10
Código-fonte 6 - Acesso aos elementos de um *array* 10
Código-fonte 7 - Indexação com início, fim e passo 11
Código-fonte 8 - Operações aritméticas com *arrays* 12
Código-fonte 9 - Exemplos de funções matemáticas 13
Código-fonte 10 - Transposição de uma matriz, ou seja, inversão de suas linhas e colunas 14
Código-fonte 11 - Produto matricial 15
Código-fonte 12 - Determinante e inversão de matriz 15
Código-fonte 13 - Salvando e carregando *arrays* em arquivos binários 16
Código-fonte 14 - Salvando e carregando *arrays* em arquivos de texto 17
Código-fonte 15 - Processamento de estatístico descritivo por eixo e total 19
Código-fonte 16 - Resolução de sistemas lineares 19
Código-fonte 17 - Conversão de imagem para escala de cinza 20

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE COMANDOS DE PROMPT

Comando de prompt 1 - Comando para instalação da biblioteca.....7

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 A MAGIA DA MATEMÁTICA NO PYTHON 6
1.1 Introdução 6
1.2 Origem e evolução 6
1.3 Instalação 7
1.4 Como utilizar 7

2 ESTRUTURA E PRINCIPAIS FUNÇÕES 8
2.1 Criação de *arrays* 8
2.2 Acesso e manipulação de elementos 10

3 PROCESSAMENTO ARITMÉTICO 12
3.1 Operações básicas 12
3.2 Funções matemáticas 12

4 OPERAÇÕES MATRICIAIS 14
4.1 Produto matricial 14
4.2 Determinante e inversa 15

5 MANIPULANDO ARQUIVOS 16
5.1 Salvando e carregando *arrays* em arquivos binários 16
5.2 Salvando e carregando em formato de texto 16

6 APLICAÇÕES PRÁTICAS 18
6.1 Análise de dados 18
6.2 Resolução de sistemas lineares 19
6.3 Processamento de imagens digitais 20

CONCLUSÃO 22

REFERÊNCIAS 23

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 A MAGIA DA MATEMÁTICA NO PYTHON

1.1 Introdução

A biblioteca Numerical Python (NumPy) exerce um papel fundamental para a computação científica por meio da linguagem Python, conhecida por sua eficiência, flexibilidade e facilidade de implementação. Com *arrays* poderosos e funções matemáticas otimizadas, ela facilita a realização de cálculos complexos de maneira rápida e intuitiva. NumPy é amplamente utilizado em diversas áreas envolvendo análise de dados e cálculos matriciais, integrando-se perfeitamente com outras bibliotecas populares, como Pandas, SciPy, Matplotlib e TensorFlow. Recentemente, a biblioteca foi atualizada para a versão 2.0, trazendo avanços significativos que aprimoram ainda mais essas funcionalidades e solidificam seu papel fundamental no desenvolvimento de soluções computacionais robustas e eficientes (NumPy, 2024a).

1.2 Origem e evolução

Desenvolvida por Travis Oliphant em 2005, a biblioteca Numerical Python consolidou duas bibliotecas anteriores, conhecidas como Numeric e Numarray, que já forneciam funcionalidades básicas para manipulação de *arrays* em Python. Travis, ao reconhecer a necessidade de uma biblioteca mais robusta e eficiente, desenvolveu o NumPy para atender à crescente demanda por ferramentas de computação científica. Desde então, a biblioteca está em evolução contínua, recebendo contribuições de uma comunidade ativa de desenvolvedores e cientistas de dados.

A atualização mais recente para a versão 2.0 trouxe avanços significativos. Entre as melhorias estão a otimização de desempenho para arquiteturas modernas, suporte melhorado para tipos de dados adicionais e aprimoramentos na interface de programação, tornando-a mais intuitiva e fácil de usar. Essas atualizações não apenas aumentam a eficiência da computação numérica, mas também facilitam a integração de NumPy com tecnologias emergentes (NumPy, 2024b).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.3 Instalação

A biblioteca NumPy já vem instalada por padrão com a distribuição Anaconda. Porém, você também pode utilizar o gerenciador de pacotes *pip* para realizar uma instalação independente via terminal com o seguinte comando:

```
pip install numpy
```

Comando de prompt 1 - Comando para instalação da biblioteca
Fonte: Elaborado pelo autor (2024)

1.4 Como utilizar

Para começar a utilizar o NumPy, é necessário importar a biblioteca em seu script Python. A convenção é importá-la seguida pelo codinome “np”, facilitando o acesso posterior às suas funções e métodos, como especificado a seguir:

```
import numpy as np
```

Código-fonte 1 - Exemplo de declaração da biblioteca
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 ESTRUTURA E PRINCIPAIS FUNÇÕES

O tipo de dado fundamental da biblioteca NumPy é o ndarray, ou *N-dimensional array*, que permite a criação e manipulação de estruturas unidimensionais ou multidimensionais de maneira eficiente. Os *arrays* do NumPy são mais compactos e rápidos em comparação com as listas nativas do Python, permitindo que as operações vetorizadas evitem loops explícitos, aperfeiçoando assim o desempenho computacional. Além disso, o NumPy fornece uma vasta gama de recursos matemáticos avançados, como funções trigonométricas, exponenciais, logarítmicas, funções de arredondamento e estatísticas, como média, mediana, variância e desvio padrão, que são altamente otimizadas para execução rápida e precisa.

2.1 Criação de *arrays*

Arrays são estruturas de dados fundamentais no NumPy e podem ser construídos de forma unidimensional, como vetores; bidimensional, como matrizes; ou até mesmo multidimensional, permitindo a organização de elementos numéricos ou mistos. A forma unidimensional é o tipo mais comum e adequado para execução de operações matemáticas. Os *arrays* são mais eficientes em termos de armazenamento e processamento comparado às listas nativas do Python, graças ao uso de técnicas como alocação contínua de memória e operações vetorizadas.

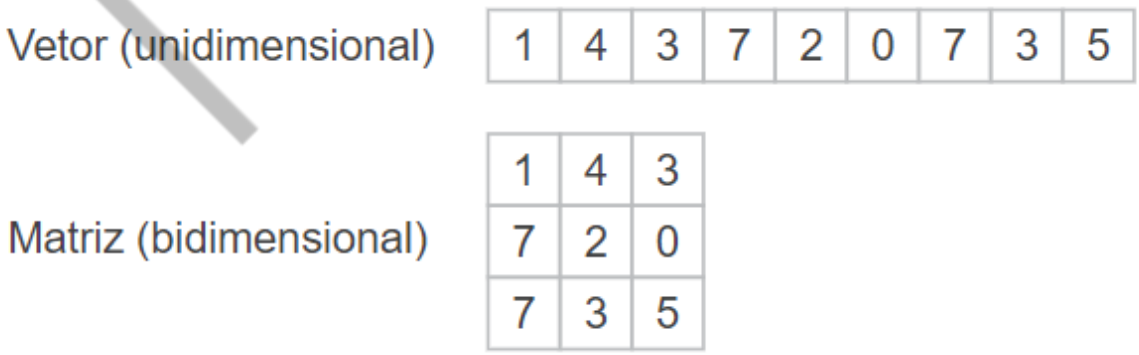

Figura 1 - Representação de vetores e matrizes
Fonte: Elaborado pelo autor (2024)

O comando *np.array* converte uma lista nativa do Python em um *array* NumPy, permitindo a utilização das operações otimizadas da biblioteca.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exemplo:

```
# Array unidimensional a partir de uma lista
a = np.array([1, 4, 3, 7, 2, 0, 7, 3, 5])

# Array bidimensional a partir de uma lista
b = np.array([[1, 4, 3], [7, 2, 0], [7, 3, 5]])
```

Código-fonte 2 - Exemplo de criação de *arrays* a partir de uma lista
Fonte: Elaborado pelo autor (2024)

Para criação de *arrays* a partir do seu formato, ou seja, quantidade de linhas e colunas, utilizam-se os seguintes métodos:

```
# Array com 3 linhas e 4 colunas
# Preenchido com números zero
c = np.zeros((3, 4))

# Array com 2 linhas e 3 colunas
# Preenchido com números um
d = np.ones((2, 3))

# Array com 2 linhas e 2 colunas
# Preenchido com valores aleatórios entre 0 e 1
e = np.random.random((2, 2))
```

Código-fonte 3 - Exemplo de criação de *arrays* a partir do formato
Fonte: Elaborado pelo autor (2024)

Além disso, existem funções que atendem situações específicas, como valores sequenciais ou distribuição linear.

```
# Array com uma sequência de números
e = np.arange(0, 10, 2

# Array com valores espaçados linearmente
f = np.linspace(0, 1, 5)
```

Código-fonte 4 - Exemplo de criação de *arrays* com distribuições específicas
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.2 Acesso e manipulação de elementos

Acessar e manipular os elementos de um *array* é essencial para realizar operações em subconjuntos de dados, facilitando análises e transformações. Com a biblioteca NumPy esse processo é executado através de indexadores numéricos, iniciados em zero, de forma similar à manipulação de listas nativas do Python.

```
a = np.array([1, 4, 3, 7, 2, 0, 7, 3, 5])
b = np.array([[1, 4, 3], [7, 2, 0], [7, 3, 5]])

# Verifica o formato das matrizes
print(a.shape, b.shape) # Saída: (9,) (3, 3)

# Acesso a elementos únicos e sequenciais
print(a[0]) # Saída: 1
print(a[1:3]) # Saída: [4 3]

# Acesso à arrays bidimensionais
print(b[0]) # Saída: [1 4 3]
print(b[1, 1]) # Saída: 2
```

Código-fonte 5 - Acesso aos elementos de um *array*
Fonte: Elaborado pelo autor (2024)

```
# Modificação do elemento na posição 0
a[0] = 10
print(a) # Saída: [10 4 3 7 2 0 7 3 5]

# Modificação do elemento na posição central da matriz 3x3
b[1][1] = 10
print(b)
# Saída:
# [[1 4  3]
#  [7 10 0]
#  [7 3  5]]
```

Código-fonte 6 - Acesso aos elementos de um *array*
Fonte: Elaborado pelo autor (2024)

Assim como nas listas nativas do Python, os *arrays* do NumPy também aceitam valores negativos para inverter a ordem de leitura e até três indexadores de seleção, separados pelo símbolo de dois pontos, para especificar o início, o fim e o passo, permitindo uma manipulação ainda mais precisa da estrutura de dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

array[início:fim:passo]

Figura 2 - Anatomia dos indexadores
Fonte: Elaborado pelo autor (2024)

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])

# Seleção entre a posição 1 e 7, pulando a de 2 em 2.
# Ou seja, nesse caso todos os pares menores que 7.
print(a[1:7:2]) # Saída: [2 4 6]

# Seleção com indexador negativo
# Passo invertido, pulando de 3 em 3.
print(a[::-3]) # Saída: [9 6 3]

b = np.array([
    [1, 2, 3, 4, 5],
    [2, 3, 4, 5, 6],
    [3, 4, 5, 6, 7],
    [4, 5, 6, 7, 8],
    [5, 6, 7, 8, 9]
])

# Seleciona as linhas e colunas pulando de 2 em 2.
# Ou seja, a primeira, segunda e terceira de cada eixo.
print(b[::2, ::2]) # Saída [[1 3 5]
                            [3 5 7]
                            [5 7 9]]
```

Código-fonte 7 - Indexação com início, fim e passo
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 PROCESSAMENTO ARITMÉTICO

Assim como na matemática elementar, a biblioteca NumPy dispõe de recursos essenciais para a manipulação de números visando obter novos valores, cujo resultado se baseia na relação entre os números envolvidos na operação.

3.1 Operações básicas

NumPy permite realizar operações algébricas básicas entre *arrays* de forma simples, aproveitando-se da eficiência computacional da biblioteca. Operações como adição, subtração, multiplicação e divisão são realizadas elemento a elemento de forma encapsulada, porém, intuitiva para o programador. Essa abordagem vetorizada não só simplifica o código, mas também melhora significativamente o desempenho, eliminando a necessidade de loops explícitos no script Python.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Adição
c = a + b

# Subtração
d = a - b

# Multiplicação
e = a * b

# Divisão
f = a / b

print(c, d, e, f)
# Saída: [5 7 9] [-3 -3 -3] [4 10 18] [0.25 0.4 0.5]
```

Código-fonte 8 - Operações aritméticas com *arrays*
Fonte: Elaborado pelo autor (2024)

3.2 Funções matemáticas

Funções matemáticas são expressões que descrevem uma relação entre um conjunto de entradas e um conjunto de possíveis saídas. Elas são fundamentais em

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

diversas áreas de aplicação, tanto no contexto científico quanto corporativo, fornecendo uma maneira de modelar e resolver problemas que envolvem quantidades variáveis. Em termos simples, uma função matemática associa cada elemento de um conjunto de entrada a exatamente um elemento de um conjunto de saída. Além das operações básicas, o NumPy fornece funções matemáticas que executam cálculos elemento a elemento, tornando o processo computacional mais simples e direto. Assim como as operações aritméticas fundamentais, essas funções precisam ser otimizadas para trabalhar com *arrays* de forma eficiente.

```
a = np.array([1, 2, 3])

# Seno
b = np.sin(a)

# Cosseno
c = np.cos(a)

# Logaritmo
d = np.log(a)

# Exponencial
e = np.exp(a)

# Raiz Quadrada
e = np.sqrt(a)

print(b, c, d, e, f)]
# Saída: [0.84147098 0.90929743 0.14112001]
         [0.54030231 -0.41614684 -0.9899925]
         [0. 0.69314718 1.09861229]
         [1. 1.41421356 1.73205081]
         [0.25 0.4 0.5]
```

Código-fonte 9 - Exemplos de funções matemáticas
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 OPERAÇÕES MATRICIAIS

Operações matriciais são cálculos matemáticos específicos, essenciais para solucionar sistemas de equações lineares, executar transformações geométricas e realizar análises estatísticas multivariadas de forma mais prática e objetiva. As matrizes são estruturas que representam uma generalização dos *arrays* para duas ou mais dimensões, e elas estão presentes em diversas aplicações como álgebra linear, processamento de imagens, jogos digitais, modelagem 3D, entre outras.

```
# Criação de uma matriz 2x2
a = np.array([[1, 2], [3, 4]])

# Transposição de uma matriz
b = a.T
```

Código-fonte 10 - Transposição de uma matriz, ou seja, inversão de suas linhas e colunas
Fonte: Elaborado pelo autor (2024)

4.1 Produto matricial

O produto matricial, diferentemente da multiplicação direta, elemento a elemento, refere-se a uma operação matemática entre duas matrizes, com regras específicas, onde os valores da matriz resultante são obtidos pela soma dos produtos dos valores correspondentes das linhas da primeira matriz e das colunas da segunda matriz, conforme ilustrado na figura “Formalização do produto matricial”.

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}_{3x2} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}_{2x2} = \begin{bmatrix} a_{11} \cdot b_{11} + a_{12} \cdot b_{21} & a_{11} \cdot b_{12} + a_{12} \cdot b_{22} \\ a_{21} \cdot b_{11} + a_{22} \cdot b_{21} & a_{21} \cdot b_{12} + a_{22} \cdot b_{22} \\ a_{31} \cdot b_{11} + a_{32} \cdot b_{21} & a_{31} \cdot b_{12} + a_{32} \cdot b_{22} \end{bmatrix}_{3x2}$$

Figura 3 - Formalização do produto matricial
Fonte: Mundo Educação UOL (2024)

Em Python, com auxílio da biblioteca NumPy, este tipo de operação é realizado utilizando a função *np.dot* ou o operador @.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

# Produto matricial entre a e b
c = np.dot(a, b)

# Ou utilizando o operador @
d = a @ b

# Valor da matriz c é igual a matriz d
print(c == d) # Saída: [[ True  True]
                        [ True  True]]
```

Código-fonte 11 - Produto matricial
Fonte: Elaborado pelo autor (2024)

4.2 Determinante e inversa

O determinante e a inversa de uma matriz são conceitos fundamentais em álgebra linear, sendo utilizados para resolver sistemas de equações e entender propriedades de transformações lineares. O determinante é um valor escalar que pode indicar se uma matriz é invertível, caso seu valor seja diferente de zero. A matriz inversa, por sua vez, é utilizada para resolver equações do tipo $Ax = b$.

```
a = np.array([[1, 2], [3, 4]])

# Determinante
det = np.linalg.det(a)

# Inversa
inv = np.linalg.inv(a)
```

Código-fonte 12 - Determinante e inversão de matriz
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 MANIPULANDO ARQUIVOS

Ler e escrever dados de maneira rápida e eficiente diretamente em arquivos otimiza o fluxo de trabalho, especialmente quando se lida com grandes volumes de dados, tornando essa uma funcionalidade do NumPy indispensável.

5.1 Salvando e carregando *arrays* em arquivos binários

A capacidade de salvar e carregar *arrays* possibilita a persistência dos dados processados em disco, ou seja, seu armazenamento para consulta posterior facilita a continuidade de trabalhos em desenvolvimento e a reprodução de experimentos. O NumPy oferece funções para salvar os *arrays* em arquivos binários, garantindo eficiência na alocação de espaço e no tempo de leitura. No entanto, este formato é ilegível para os seres humanos e só pode ser manipulado por scripts Python executados no sistema operacional em que foram originalmente gerados.

```
a = np.array([1, 2, 3, 4, 5])

# Salvando um array em um arquivo binário
np.save('meu_array.npy', a)

# Carregando um array de um arquivo binário
b = np.load('meu_array.npy')

print(b) # Saída: [1 2 3 4 5]
```

Código-fonte 13 - Salvando e carregando *arrays* em arquivos binários
Fonte: Elaborado pelo autor (2024)

5.2 Salvando e carregando em formato de texto

A biblioteca NumPy também permite salvar e carregar *arrays* em arquivos de texto, oferecendo opções para definir o delimitador, a codificação e o formato dos dados, além de outros parâmetros mais específicos. Dessa forma é possível manipular os tradicionais arquivos CSV e suas variações.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
a = np.array([9, 8, 7, 6, 5, 4, 3, 2, 1])

# Salvando um array em um arquivo de texto
np.savetxt('meu_array.txt', a)

# Carregando um array de um arquivo de texto
b = np.loadtxt('meu_array.txt')

print(b) # Saída: [9. 8. 7. 6. 5. 4. 3. 2. 1.]
```

Código-fonte 14 - Salvando e carregando *arrays* em arquivos de texto
Fonte: Elaborado pelo autor (2024)

Salvar dados neste formato é útil para proporcionar a interoperabilidade com outras ferramentas, facilitando a integração entre sistemas distintos de forma assíncrona. Além disso, viabiliza a inspeção manual dos dados armazenados, pois as informações gravadas se mantêm legíveis para seres humanos, conforme ilustrado na figura a seguir, e podem ser acessadas através de editores simples como Bloco de Notas, Sublime, Notepad++, entre outros.

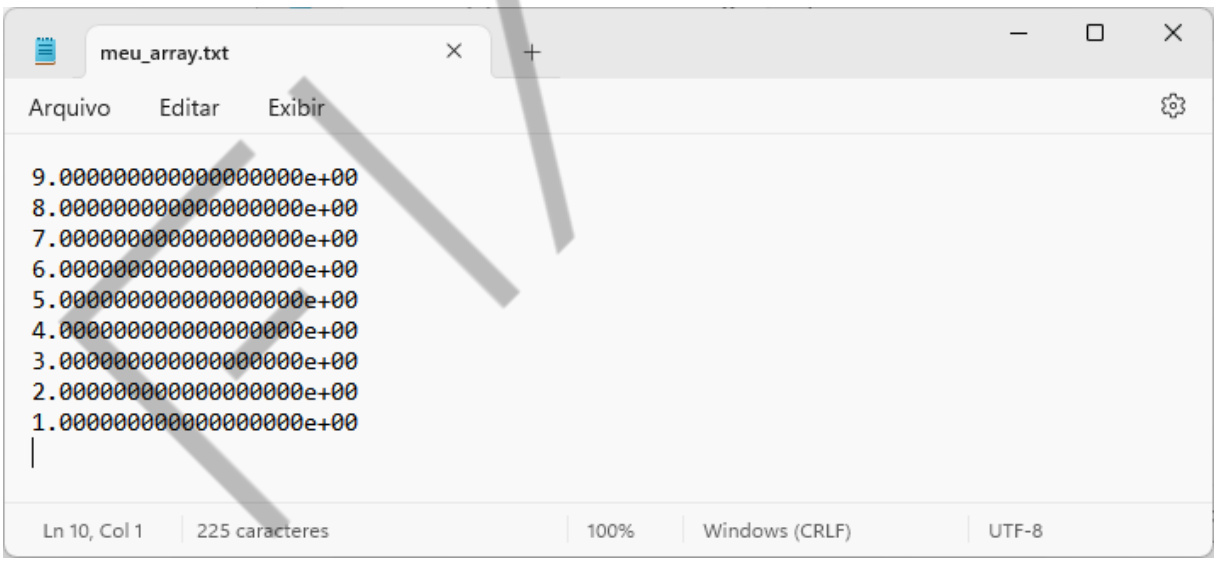

Figura 4 - *Array* armazenado em arquivo texto
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 APLICAÇÕES PRÁTICAS

A biblioteca NumPy possui aplicações práticas amplamente difundidas em diversas áreas que necessitam de soluções para computação científica e automação de processos complexos. Na Ciência de Dados e Machine Learning, os *arrays* do NumPy são usados na manipulação e análise de grandes conjuntos de informações, na preparação de amostras para classificação em modelos e na execução de cálculos matemáticos de forma eficiente.

Na engenharia, a biblioteca NumPy facilita a modelagem e simulação de sistemas físicos precisos, a análise de sinais e o processamento de imagens, onde operações matemáticas em matrizes são comuns. Em aplicações na área de finanças, é frequentemente utilizada para realizar a análise de séries temporais, os cálculos de risco, a concessão de crédito e a modelagem de portfólios. No campo da pesquisa científica, o NumPy permite a realização de experimentos computacionais automatizados, a análise estatística, a validação de hipóteses e a resolução de problemas de álgebra linear. A flexibilidade e a eficiência do NumPy fazem dele uma ferramenta versátil e essencial para tarefas que requerem manipulação intensiva de dados e cálculos numéricos robustos.

6.1 Análise de dados

O Numerical Python é amplamente utilizado em análise de dados devido à sua capacidade de manipulação de grandes conjuntos de informações, de forma rápida, precisa e confiável. A biblioteca foi projetada para trabalhar com *arrays* multidimensionais, mais compactos e eficientes que as listas nativas do Python, viabilizando o armazenamento e processamento eficaz no contexto de Big Data. Em atividades relacionadas à Ciência de Dados e Econometria, a capacidade do NumPy de realizar operações complexas de forma otimizada é fundamental.

Na prática, o NumPy permite a execução de análises elaboradas ao se integrar com outras bibliotecas como Pandas e SciPy. Isso amplia suas capacidades para tarefas que vão desde a limpeza e organização de dados até a modelagem estatística avançada, para aplicações de inteligência artificial.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Simulação de dados de vendas com dados aleatórios
# Os resultados deverão mudar a cada execução
vendas = np.random.randint(100, 200, size=(30, 4))

# Média de vendas por semana
media_semanal = np.mean(vendas, axis=0)

# Menores vendas por semana
menor_semanal = np.min(vendas, axis=0)

# Total de vendas por mês
total_mensal = np.sum(vendas)

# Maior venda do mês
maior_mensal = np.max(vendas)

print(media_semanal) # Saída: [149.76 154.03 161.26 148.8]
print(menor_semanal) # Saída: [101 100 112 105]
print(total_mensal) # Saída: 18416
print(maior_mensal) # Saída: 199
```

Código-fonte 15 - Processamento de estatístico descritivo por eixo e total
Fonte: Elaborado pelo autor (2024)

6.2 Resolução de sistemas lineares

A resolução de sistemas lineares é uma aplicação crucial da álgebra linear. O NumPy fornece ferramentas adequadas para solucionar tais questões, tornando-se essencial em áreas como engenharia, física e economia. Resolver um sistema linear significa encontrar os valores das variáveis que satisfaçam todas as equações simultaneamente. Considere o sistema de equações lineares $3x + 2y = 5$ e $x + 2y = 5$ na forma matricial $Ax = B$. A função *np.linalg.solve* resolve esse sistema linear encontrando os valores de *x* que satisfazem a equação. Neste exemplo, a solução contém os valores de *x* e *y* (Tsukada, 2016).

```
# Coeficientes do sistema linear
a = np.array([[3, 2], [1, 2]])
b = np.array([5, 5])

# Resolvendo o sistema
solucao = np.linalg.solve(a, b)

print(solucao) # Saída: [0. 2.5]
```

Código-fonte 16 - Resolução de sistemas lineares
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.3 Processamento de imagens digitais

A biblioteca NumPy facilita o processamento de imagens digitais ao permitir a manipulação das imagens como *arrays*. Neste processo, cada pixel pode ser tratado como um elemento de uma matriz, e tarefas como filtragens e transformação de cores podem ser realizadas de forma muito eficiente (Dey, 2018). No entanto, essa aplicação demanda o uso de outras bibliotecas combinadas com o NumPy, como é o caso da PIL, utilizada no exemplo a seguir para fazer a leitura da imagem.

```
import numpy as np
from PIL import Image

# Carregando uma imagem e transformação em array
imagem = Image.open('imagem.jpg')
img_array = np.array(imagem)

# Convertendo para escala de cinza
img_cinza = np.dot(img_array[..., :3], [0.29, 0.58, 0.11])

# Salvando a imagem em escala de cinza
img_final = Image.fromarray(img_cinza.astype('uint8'))
img_final.save('imagem_cinza.jpg')
```

Código-fonte 17 - Conversão de imagem para escala de cinza
Fonte: Elaborado pelo autor (2024)

Figura 5 - Imagem antes e depois do processamento — autorretrato de Van Gogh
Fonte: Van Gogh (1888), adaptado por FIAP (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Com a capacidade de processar imagens como *arrays*, é possível realizar uma ampla gama de operações, incluindo ajuste de brilho, contraste, saturação, detecção de bordas e transformações geométricas como rotação, redimensionamento e recorte. Além disso, o NumPy pode ser utilizado para análises mais complexas, como segmentação de objetos, reconhecimento de padrões e aprimoramento de imagem. Essas operações são essenciais em áreas como visão computacional, onde o processamento eficiente de imagens é fundamental para aplicações como reconhecimento facial, análise de imagens médicas e desenvolvimento de veículos autônomos. Ao integrar o Numerical Python com bibliotecas dedicadas como OpenCV e scikit-image, o poder do NumPy é amplificado, proporcionando uma plataforma robusta para o desenvolvimento de soluções de processamento de imagens digitais (Dey, 2018).

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

A biblioteca NumPy, desde sua criação, transformou a maneira como a computação numérica é realizada. Sua eficiência, flexibilidade e facilidade de uso a tornaram uma ferramenta indispensável para cientistas de dados, engenheiros, pesquisadores e desenvolvedores em todo o mundo. Com a recente atualização para a versão 2.0, o NumPy continua em evolução, solidificando cada vez mais seu papel de destaque na computação científica e no desenvolvimento de soluções computacionais avançadas.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

DEY, Sandipan. **Hands-On Image Processing with Python.** Baltimore: Packt Publishing Limited, 2018.

GOGH, Vincent Van. **Autorretrato com Chapéu de Feltro**. 1888. Pintura, óleo sobre tela, 47 x 37.5 cm.

NUMPY user guide. **NumPy.org**, 2024a. Disponível em: <http://www.numpy.org>. Acesso em: 13 ago. 2024.

NUMPY 2.0.0 Release Notes. **NumPy.org**, 2024b. Disponível em: <https://numpy.org/devdocs/release/2.0.0-notes.html>. Acesso em: 13 ago. 2024.

RIZZO, M. L. A. Multiplicação de matrizes. **Mundo Educação UOL**, [*s. d.*]. Disponível em: <https://mundoeducacao.uol.com.br/matematica/multiplicacao-matrizes.htm>. Acesso em: 15 ago. 2024.

TSUKADA, M. *et al*. **Linear Algebra with Python**. Springer, 2016.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com